Περιγραφή Υλοποίησης & Οδηγίες Χρήσης «Τα Μήλα»

Ομάδα: 3 άτομα

Έγγραφο Σχεδιασμού & Υλοποίησης: Project "Μήλα"

1. Επισκόπηση Παιχνιδιού

1.1 Concept Το Project "Μήλα" είναι ένα 3D Action/Arcade παιχνίδι σε μια κλειστή αρένα. Ο παίκτης καλείται να επιβιώσει και να νικήσει τα ζόμπι, χρησιμοποιώντας μπάλες ως όπλο. Το παιχνίδι διαθέτει ένα σύστημα προοδευτικής δυσκολίας με 3 επίπεδα (levels), όπου οι εχθροί (NPCs) αποκτούν σταδιακά νέες, πιο έξυπνες ικανότητες.

1.2 Στόχος (Objective) Ο κύριος στόχος του παίκτη είναι να καταστρέψει όλα τα ζόμπι σε κάθε ένα από τα τρία levels. Η νίκη επιτυγχάνεται με την ολοκλήρωση του τρίτου και δυσκολότερου level.

1.3 Κοινό-Στόχος (Target Audience) Το παιχνίδι απευθύνεται σε casual παίκτες που απολαμβάνουν παιχνίδια με γρήγορη δράση, απλούς κανόνες και έμφαση στα αντανακλαστικά και τη στρατηγική.

Το παιχνίδι υλοποιήθηκε σε Unity 2022.3.55f1

2. Πώς Παίζεται το Παιχνίδι

2.1 Χειρισμός (Controls) Ο χειρισμός του παίκτη είναι απλός και άμεσος:

- **Κίνηση:** Πλήκτρα **WASD** για κίνηση μπρος, πίσω, αριστερά και δεξιά.
- **Πιάσιμο Μπάλας (Catch): Δεξί Κλικ** του ποντικιού. Ο παίκτης προσπαθεί να πιάσει μια μπάλα που βρίσκεται κοντά του.
- **Πέταγμα Μπάλας (Throw): Αριστερό Κλικ** του ποντικιού (λειτουργεί μόνο όταν ο παίκτης κρατάει ήδη μπάλα).

2.2 Βασικοί Κανόνες Ο παίκτης ξεκινάει σε μια καθορισμένη ζώνη της αρένας. Ο στόχος είναι να χρησιμοποιήσει τις διαθέσιμες μπάλες για να χτυπήσει και να εξοντώσει όλα τα ζόμπι που βρίσκονται στην κεντρική ζώνη.

- **Συλλογή Μπαλών:** Ο παίκτης πρέπει να πλησιάσει τις μπάλες που βρίσκονται σε ένα συγκεκριμένο σημείο ανεφοδιασμού στη ζώνη του και να τις πιάσει με το δεξί κλικ.
- **Στόχευση:** Όταν ο παίκτης κρατάει μπάλα, εμφανίζεται ένα λέιζερ στόχευσης. Το λέιζερ γίνεται κόκκινο όταν "κλειδώνει" πάνω σε ένα ζόμπι, υποβοηθώντας τη σκόπευση.
- **Εξόντωση:** Ένα χτύπημα με την μπάλα είναι αρκετό για να καταστρέψει ένα ζόμπι, ανάλογα με τις αντιδράσεις του.

2.3 Σύστημα Levels Η πρόκληση αυξάνεται σταδιακά σε κάθε level:

- **Level 1 (Εύκολο):** Ο παίκτης αντιμετωπίζει έναν συγκεκριμένο αριθμό ζόμπι με περιορισμένες μπάλες. Τα ζόμπι είναι απλοί στόχοι και καταστρέφονται αμέσως με το χτύπημα.
- **Level 2 (Μέτριο):** Τα ζόμπι αποκτούν την ικανότητα **Dodge**. Πλέον, υπάρχει πιθανότητα να αντιληφθούν την μπάλα που πλησιάζει και να προσπαθήσουν να την αποφύγουν, κάνοντας την εξόντωσή τους πιο δύσκολη. Ο παίκτης έχει περισσότερες μπάλες στη διάθεσή του.
- **Level 3 (Δύσκολο):** Τα ζόμπι αποκτούν και την ικανότητα **Catch**. Έχουν πλέον την ικανότητα να πιάσουν την μπάλα του παίκτη και να δυσκολέψουν τον παίκτη που θέλει να τους εξοντώσει. Ο παίκτης έχει περισσότερες μπάλες από πριν.

2.4 Συνθήκες Τέλους Παιχνιδιού

- **Ήττα (Game Over):** Στα Levels 1 και 2, αν ο παίκτης μείνει χωρίς μπάλες (τόσο στα χέρια του όσο και στη σκηνή) και υπάρχουν ακόμα ζόμπι, εμφανίζεται μήνυμα αποτυχίας και το παιχνίδι ξεκινά από την αρχή (Level 1).
- **Νίκη Παιχνιδιού (Win):** Αν ο παίκτης καταστρέψει όλα τα ζόμπι στο Level 3, εμφανίζεται μήνυμα τελικής νίκης με επιλογή για επανεκκίνηση.

3. Περιγραφή Υλοποίησης (Implementation)

Η αρχιτεκτονική του παιχνιδιού βασίζεται σε μερικά κεντρικά scripts που συνεργάζονται μεταξύ τους.

3.1 Κεντρικός Διαχειριστής (GameManager.cs) Αυτό το script λειτουργεί ως ο "εγκέφαλος" και ο "σκηνοθέτης" του παιχνιδιού.

- **Διαχείριση Levels:** Περιέχει μια λίστα από `LevelSettings`, μια προσαρμοσμένη δομή που ορίζει τους κανόνες για κάθε level (αριθμό zombies/balls, ικανότητες AI,πιθανότητες για destroyed/catch). Η συνάρτηση `StartLevel()` είναι υπεύθυνη για το στήσιμο κάθε επιπέδου.
- **Κατάσταση Παιχνιδιού:** Παρακολουθεί συνεχώς μεταβλητές όπως `zombiesRemaining` και `activeBallsInScene` για να ελέγχει τις συνθήκες νίκης και ήττας.
- **Έλεγχος UI:** Είναι υπεύθυνος για την ενημέρωση όλων των στοιχείων του UI (μετρητές, panels νίκης/ήττας).
- **Επικοινωνία:** Χρησιμοποιεί το Singleton pattern (`GameManager.instance`) για να είναι εύκολα προσβάσιμος από όλα τα άλλα scripts.

3.2 Χειρισμός Παίκτη (PlayerController.cs) Αυτό το script είναι αποκλειστικά υπεύθυνο για τον παίκτη.

- **Κίνηση:** Η κίνηση υλοποιείται μέσω `transform.Translate`, δίνοντας άμεσο και γρήγορο έλεγχο. Ο περιορισμός του παίκτη στη ζώνη του γίνεται μέσω bake του NavMeshSurface .
- **Animations:** Η κίνηση συγχρονίζεται με τον `Animator` μέσω της παραμέτρου `moveSpeed` (Float), ενώ οι δράσεις (όπως η ρίψη) ενεργοποιούνται μέσω `Triggers` (`isThrowing`).

- **Δράσεις (Catch/Throw):** Το "catch" υλοποιείται με την προσωρινή ενεργοποίηση ενός `Trigger Collider` (`CatchZone`). Το "throw" καλείται από ένα `Animation Event` για τέλειο συγχρονισμό με την κίνηση.
- **Στόχευση:** Το λέιζερ υλοποιείται με ένα `Line Renderer`. Η λογική του **Aim Assist** χρησιμοποιεί `Physics.OverlapSphere` για να βρει όλους τους εχθρούς σε μια ακτίνα και μετά υπολογίζει με το `Vector3.Angle` ποιος είναι ο πιο κεντρικός στόχος μπροστά από τον παίκτη.

3.3 Τεχνητή Νοημοσύνη (CompleteZombieAI.cs & SimpleAISpawner.cs)

- **`SimpleAISpawner.cs`:** Ένα script με μοναδικό σκοπό τη δημιουργία (`Instantiate`) του σωστού αριθμού zombies στην αρχή κάθε level, κατόπιν εντολής του `GameManager`.
- **`CompleteZombieAI.cs`:** Το "μυαλό" κάθε ζόμπι.
    - **Διαμόρφωση:** Διαθέτει μια δημόσια συνάρτηση `ConfigureAI(bool allowsDodge, bool allowsCatch)`, την οποία καλεί ο `GameManager` για να "ενεργοποιήσει" ή να "απενεργοποιήσει" τις ικανότητες του ζόμπι ανάλογα με το level.
    - **Κίνηση:** Βασίζεται στο σύστημα **NavMesh** της Unity. Ένα component `NavMeshAgent` του επιτρέπει να κινείται αυτόνομα πάνω στον προ-υπολογισμένο "χάρτη" (NavMesh), μένοντας πάντα μέσα στην καθορισμένη του ζώνη (`Area Mask`).
    - **Αντίληψη (Dodge):** Χρησιμοποιεί έναν μεγάλο `Sphere Collider` ρυθμισμένο ως `Trigger`. Η συνάρτηση `OnTriggerEnter` ανιχνεύει τις μπάλες που πλησιάζουν και, βάσει πιθανότητας, ενεργοποιεί την αποφυγή.
    - **Αντίδραση (Catch):** Χρησιμοποιεί έναν `Capsule Collider` (όχι trigger) για τη φυσική σύγκρουση. Η συνάρτηση `OnCollisionEnter` αποφασίζει, βάσει πιθανότητας, αν το ζόμπι θα πιάσει την μπάλα ή θα καταστραφεί.

3.4 User Interface (UI)

- Όλα τα στοιχεία του UI βρίσκονται κάτω από ένα μοναδικό **`Canvas`**.
- Για την εμφάνιση κειμένου χρησιμοποιείται το πακέτο **`TextMeshPro`** για καλύτερη ποιότητα.
- Τα κουμπιά (`Button`) στα panels νίκης/ήττας συνδέονται μέσω του συστήματος **`OnClick()`** του Inspector για να καλούν τις δημόσιες συναρτήσεις του `GameManager` (π.χ., `GoToNextLevel()`, `RestartGame()`).